

Evolutionary Portfolio Optimisation

*Exploring Evolutionary Algorithm Design with a
Multi-Objective Optimisation problem in Finance.*

35092 – Topics in Computational Modelling:
From Information Theory to Evolutionary Algorithms.

Marco Lomele, 3152128

May 2024

Table of Contents

PORTFOLIO OPTIMISATION	2
FORMAL DEFINITION	2
PROBLEM DEFINITION	3
PURPOSE	3
PROCESS	4
ALGORITHM DESIGN	5
POPULATION INITIALISATION	6
FITNESS FUNCTION	7
EVOLUTIONARY STRATEGY	8
SELECTION	9
CROSSOVER	10
MUTATION	11
RESULTS	12
STEP 1 – SELECTION MECHANISMS	12
STEP 2 – CROSSOVER AND MUTATION MECHANISMS	13
FUTURE RESEARCH	15
BIBLIOGRAPHY	16
APPENDIX	16

Portfolio Optimisation

Imagine you win 1 million dollars from the lottery, and you don't know what to buy with them. One possibility is to invest in the stock market. This would protect the prize from inflation, and with the right choice of stocks, you could also make extra money.

Portfolio optimisation deals with the problem of choosing which stocks to invest in. A stock is a share in the ownership of a company. Investors can buy multiple shares of the same company, increasing their ownership. A portfolio then is the collection of shares of different companies that an investor owns.

Portfolio optimisation is a type of backward financial analysis strategy because it finds a solution for data from the past. Using these results to make present investment decisions would rely on the assumption that on average, history repeats itself.

Formal Definition

We borrow the framework developed by Guarino A. et al. in 2024 [1]. Take a set of N stocks, $A_1, \dots, A_N$, and choose a time window (e.g. 1 year). For each stock A_i , take its daily prices C_i , and compute its daily returns $r_{i,t}$:

$$r_{i,t} = \ln \frac{c_{i,t}}{c_{i,t-1}}$$

where $c_{i,t}$ is the price of stock i on day t , for $t = 1, \dots, T$ and $i = 1, \dots, N$. We collect all the daily returns of stock i into a list $TR_i = [r_{i,1}, \dots, r_{i,T}]$.

Next, we define a portfolio P as a vector $P = [P_1, \dots, P_N]$, where $P_i \in \mathbb{R}^+$ is an integer indicating the number of shares of stock A_i purchased. From an evolutionary perspective, P_i represents gene i of P . A portfolio has the following measures:

- overall price: $C(P, T) = np.dot(P, c_T)$, where $c_T \in \mathbb{R}^N$ are all the prices of the stocks at time T .
- expected returns: $R(P) = np.dot(P, \bar{r})$, where $\bar{r} \in \mathbb{R}^N$ holds the mean of the returns for each stock.

- expected variance: $V(P) = \sum_{A_i, A_j | i \neq j} \mathcal{V}(A_i, A_j)$, which is the sum of pairwise variances:

$$\mathcal{V}(A_i, A_j) = \frac{P_i}{\sum_{k=1}^N P_k} \times \sigma_{TR_i} + \frac{P_j}{\sum_{k=1}^N P_k} \times \sigma_{TR_j} + 2 \times \frac{P_i}{\sum_{k=1}^N P_k} \times \frac{P_j}{\sum_{k=1}^N P_k} \times \mathcal{C}_{TR_i TR_j}$$

Here, σ_{TR_i} represents the variance of TR_i , and $\mathcal{C}_{TR_i TR_j}$ is the covariance between TR_i , all the returns of stock i , and TR_j , all the returns of stock j .

Finally, the quality of portfolio P is then measured in terms of its expected returns $R(P)$ and its expected variance $V(P)$, with higher values of $R(P)$ and lower values of $V(P)$ being better. In finance, expected variance is often called risk.

Problem Definition

Given a set of stocks $A_1, \dots, A_N$ and a budget B , the portfolio optimisation problem is:

$$\begin{aligned} & \max_{P_i, i \in [1, N]} R(P) \\ & \min_{P_i, i \in [1, N]} V(P) \\ & s. t. C(P, T) \leq B \end{aligned}$$

We seek to find the choice of stocks that maximise the portfolio's expected returns, minimise its expected variance, and fit the budget constraint. Given that this is a multi-objective optimisation problem, we look for a set of non-dominated solutions.

For our investigation, we start with a budget of \$1 million and use 6 years of daily price data between 2018 and 2024 of the top 210 stocks from the S&P 500.

Purpose

Portfolio Optimisation has been studied by many because its model is simple and applicable to the real world. In 1952, Markowitz proposed the first version, one without a budget constraint and a closed-form solution. With budget constraint, the optimisation becomes non-convex.

Luckily, Evolutionary Algorithms (EAs) are known to be effective in solving complex multi-objective optimisation problems. However, research like [1] is scarce, vague on which experiment conditions have been used, such as the number of stocks, and unclear on explaining which algorithms have been tested and why.

The purpose of this investigation is therefore to study algorithm design in the realm of multi-objective optimisation problems using budget-constrained portfolio optimisation.

Process

We will test algorithm designs along three dimensions: selection, crossover, and mutation. Through the information flow, the quality of the solutions will act as a signal for the quality of the design choice. We quantify the quality of a solution by computing the area under the curve (AUC) of its Pareto frontier.

Step 1

Compare three selection mechanisms: Double Tournament, Non-dominated Sorting Genetic Algorithm 2 (NSGA2), Strength Pareto Evolutionary Algorithm 2 SPEA2. The other two dimensions will be kept constant to One Point crossover and Uniform Integer mutation, the set-up used by the authors in [1]. We start with selection because population diversity and selective pressure are the only two primary factors in genetic search (Whitley, 1989) [2].

For each selection mechanism, we compute 10 Pareto frontiers with different random initialisations, get their respective areas under the curve (AUC), compute the average. The best selection mechanism will be the one with the highest AUC value.

Step 2

Using the best selection mechanism found in step 1, test two additional crossover mechanisms, Blend and Uniform, and two additional mutation mechanisms, Gaussian and Polynomial Bounded. Including the operators used in [1], we have a total of 9 different algorithm designs.

For each permutation, we repeat the same procedure; generate 10 Pareto frontiers with different random initialisations, find the average AUC, and choose the configuration with the highest score.

Algorithm Design

Below is a table summarising the various design choices that will be tested.

Representation	Lists of integers.
Crossover	One-point, Blend, Uniform.
Crossover probability	50% overall, 10% within individual.
Mutation	Uniform Integer, Gaussian, Polynomial Bounded.
Mutation probability	50% overall, 10% within individual.
Parent selection	Double Tournament, NSGA2, SPEA2.
Survival selection	Double Tournament, NSGA2, SPEA2.
Population size	100
Number of offspring	100
Initialisation	Random with budget constraint.
Number of generations	50
Termination condition	Reaching 50 generations.

Population Initialisation

Each individual in the population is a portfolio P_k . It is represented as a sequence of genes $[P_{k,1}, \dots, P_{k,N}]$, where each gene $P_{k,i}$ indicates the amount of shares of stock i portfolio k purchases. $i = 1 \dots N$ is the number of stocks considered and $k = 1, \dots, K$ is the size of the population. In our case, $N = 210$ and $K = 100$.

Because exploration is more important at the beginning of an evolutionary search process, it is preferred to initialise highly diverse individuals, especially in multi-objective optimisation problems. This diversity allows us to sample distant regions of the solution space. We achieve this feature by injecting randomness in choosing what stocks to buy and choosing how many shares of each stock to buy.

Inspired by the procedure proposed in [1], the initialisation algorithm gradually and stochastically builds portfolio configurations. Specifically, it starts from an empty portfolio, randomly chooses how many unique stocks to buy, and then buys a random amount of shares per stock, always checking that the total cost is less than the budget.

```
def generate_individual():
    max_budget = budget + 1
    while max_budget > budget:
        ind = [0 for i in range(num_assets)]
        stocks_to_buy = np.random.randint(0, num_assets)
        stocks_bought = []
        current_budget = budget

        for i in range(stocks_to_buy):
            stock_to_buy = np.random.randint(0, num_assets)
            while stock_to_buy in stocks_bought:
                stock_to_buy = np.random.randint(0, num_assets)

            stocks_bought.append(stock_to_buy)
            upperbound_default =
                int(current_budget/current_prices[stock_to_buy])
            if upperbound_default > 1:
                ind[stock_to_buy] = np.random.randint(1, upperbound_default)

            current_budget = budget - np.dot(np.array(ind), current_prices)
        max_budget = np.dot(np.array(ind), current_prices)
    return ind
```

Fitness Function

We implement the max-min double-objective function described [above](#). The fitness of each candidate solution is a tuple holding two numbers: the expected returns and expected variance.

```
def compute_exp_variance(individual):
    relative_amounts = individual / np.sum(individual)
    w_var = daily_returns_var * relative_amounts
    i_indices, j_indices = np.meshgrid(np.arange(num_assets),
                                       np.arange(num_assets),
                                       indexing='ij')

    cov_terms = 2 * relative_amounts[i_indices] *
                relative_amounts[j_indices] *
                daily_returns_covmat[i_indices, j_indices]

    off_diagonal_mask = 1 - np.eye(num_assets, num_assets)
    overall_exp_variance = np.sum((w_var[:, None] + w_var[None, :] +
                                   cov_terms) * off_diagonal_mask)

    return overall_exp_variance

def evaluate(individual):
    individual = np.array(individual)
    exp_variance = compute_exp_variance(individual)
    exp_return = np.dot(individual, daily_returns_mean)

    return exp_variance, exp_return
```

Utilities

The fitness evolves solutions towards two unwanted scenarios: (1) a portfolio that costs more than the budget constraint, and (2) a portfolio with negative-valued entries. We implement two functions during crossover and mutation to deal with these issues: `check_budget_constraint` and `check_negatives`. The former is written below. The latter simply runs a forward pass of the individual and sets any negative values to 0.

```
def check_budget_constraint(individual, delta = 1, power = 1):
    individual_np = np.array(individual, dtype=int)
    while np.dot(individual_np, current_prices) > budget:
        possible_indices = np.where(individual_np > 0)[0]
        index_count = min(power, len(possible_indices))
        index = np.random.choice(possible_indices, index_count,
                                replace=False)
        individual_np[index] -= delta

    for i in range(len(individual)):
        individual[i] = int(individual_np[i])
    return individual
```

Evolutionary Strategy

We choose the “Mu plus Lambda” ($\mu + \lambda$) strategy over the alternative “Mu comma Lambda” (μ, λ) strategy. The main difference is that the former preserves the fittest μ individuals at each generation, while the latter replaces completely parents. Also, to keep the selection pressure at a reasonable level, we set $\mu = \lambda$.

We choose “Mu plus Lambda” to prioritise robustness over quick evolution. The idea is to reduce sources of variability in the population to have more control over the evolutionary process. Exploration is not sacrificed, because at each generation novel offspring is generated through crossover and mutation. Moreover, we induce heterogeneity via highly stochastic population initialisation.

Another reason for using “Mu plus Lambda” is elitism, which means exploiting the best individuals by preserving them. Because we are searching for the Pareto front, preservation is crucial. It would be sub-optimal to replace entirely the population, as we would risk losing dominant solutions.

EA_MuPlusLambda Pseudocode

```
Evaluate individuals in the population
  For each individual:
    Calculate fitness
    Assign fitness value

If halloffame exists (e.g. Pareto front):
  Update halloffame with the population

For each generation from 1 to ngen:
  Create offspring through variation (crossover and mutation)

  Evaluate individuals in the offspring
    For each individual:
      Calculate fitness
      Assign fitness value

  If halloffame exists:
    Update halloffame with the offspring

  Select next generation population from parents + offspring

Return final population
```

Selection

In this investigation, we consider three selection mechanisms.

Double Tournament

The Double Tournament selection mechanism involves two stages: fitness and diversity. In the fitness stage, a subset of individuals is selected, and the one with the highest fitness wins. In the diversity stage, another subset is selected, and the one with the highest diversity measure, like crowding distance, wins. Finally, the winners of the two stages are compared, and the ultimate winner is chosen with some predefined criteria, often involving a balance between fitness and diversity.

In the context of multi-objective optimisation, this technique ensures a good balance between selecting individuals with high fitness and maintaining diversity in the population, i.e. a balance between exploitation and exploration. This solves the issue of single tournament selection has, which is that only a single value of fitness is considered, making it unable to compare different non-dominated solutions.

Non-dominated Sorting Genetic Algorithm 2 (NSGA2)

NSGA2 selects individuals in two stages. First, it sorts the population into non-dominated fronts. Each front holds the solutions that are dominated by the solutions in the lower fronts. The solutions in the lowest front are dominated by no one, and the solutions in the highest front are dominated by all solutions.

Next, individuals who are in the same front are ranked by crowding distance, which measures the density of solutions surrounding a particular solution, with higher distances being better to ensure diversity.

The combination between non-dominant sorting and diversity-preservation makes NSGA2 an effective selection operator for optimisation problems with multiple objectives.

Strength Pareto Evolutionary Algorithm 2 (SPEA2)

Similarly to NSGA2, SPEA2 considers dominance and diversity. First, it determines each individual's fitness by counting the number of solutions it dominates and is dominated by. Then, it estimates the density of the solutions using k-nearest neighbour, with typically $k = \sqrt{N}$. Finally, it keeps track of an archive of non-dominated individuals and updates it by selecting the fittest and least dense solutions.

By preserving diversity and recording non-dominated solutions, SPEA2 effectively manages convergence to the pareto front, making it a great candidate for multi optimisation problems. Because of the archive, SPEA2 scores equally two solutions on the pareto front. In contrast, NSGA2 does not do so.

Crossover

In this investigation, we consider three crossover mechanisms.

One Point

On-point crossover works by randomly selecting a single crossover point along the length of the parent individuals and swapping the segments of the parents' chromosomes after the crossover point.

This technique preserves the order of the genes in the parents, important because each gene indicates a different stock with unique attributes. One Point could be beneficial if there are groupings of stocks in which it is favourable to invest together.

Blend

Blend crossover generates offspring by selecting a point in a region that is slightly bigger than the N -dimensional rectangle spanned by the two parents. In our case, the scaling of the region was set to 0.5, the default value.

Blend crossover allows for fine-tuned exploration of the search space, promoting diversity and acting as a source of variation to the population. Because our goal is to search for the Pareto frontier, exploration in a narrow range is desirable.

Uniform

Uniform crossover works by randomly assigning genes from the two parents using a bit-wise mask. For this investigation, the individual probability was set to 0.5, meaning that on average, each parent contributes equally to the offspring.

This technique leads to a greater degree of mixing between parents. It relies on the assumption that there are no strong linkages among genes, which translates to investments being independent one from the other, contrasting to what One Point crossover may find.

```
def mate(ind1, ind2):
    offspring1, offspring2 = tools.cxBlend(ind1, ind2, alpha=0.5)

    offspring1 = check_negatives(offspring1)
    offspring2 = check_negatives(offspring2)

    delta, alpha = 1, 1
    offspring1 = check_budget_constraint(offspring1, delta, alpha)
    offspring2 = check_budget_constraint(offspring2, delta, alpha)

    return offspring1, offspring2
```

Mutation

We compare three different mutation mechanisms: Uniform Integer, Gaussian, and Polynomial Bounded. We modify their functions such that the mutation adapts to the available budget by setting an upper bound xu for each gene i to be mutated:

```
xu = int(current_budget / current_prices[i]) + individual[i] + push
```

where *push* promotes exploration by increasing the quantity of shares that can be bought. Budget violations will be fixed by `check_budget_constraint`. Gaussian mutation does not have a fixed range, so the number purchasable stocks is set as the standard deviation. As a source of variability, *push* is set to 10.

Uniform Integer

Uniform integer mutation replaces a gene with a random integer drawn from a uniform distribution in a specified range. This approach is useful for discrete problems like ours, where each gene is an integer value that can mutate in a predefined range. Also, Uniform Integer explores the solution space uniformly, a desirable quality in an N -dimensional space. We set a lower bound of 0 and an individual mutation probability (indpb) of 10%, such that the evolution is highly likely to add shares to portfolios.

Gaussian

Gaussian mutation adds a random value drawn from a Gaussian distribution with mean 0 and adaptive standard deviation. This mechanism induces continuous fine-tuning, crucial for testing candidates who appear to be non-dominated. Moreover, the Gaussian mutation reduces the value of a gene 50% of the time, increasing the budget available for the individual, so that other shares can be bought in the next generation. Again, we choose an indpb of 10% for robustness.

Polynomial Bounded

Polynomial bounded mutation perturbs a gene using a polynomial probability distribution. The strength of the mutation depends on a parameter called *eta*; high *eta* produces offspring similar to parents, and low *eta* gives diverging solutions. We set an *eta* of 11, a middle value, a lower bound value of 0, and the usual indpb of 10%. This approach combines the benefits from both the previous two methods; it promotes continuous exploration but restricts it in a range, resulting in effective fine-tuning.

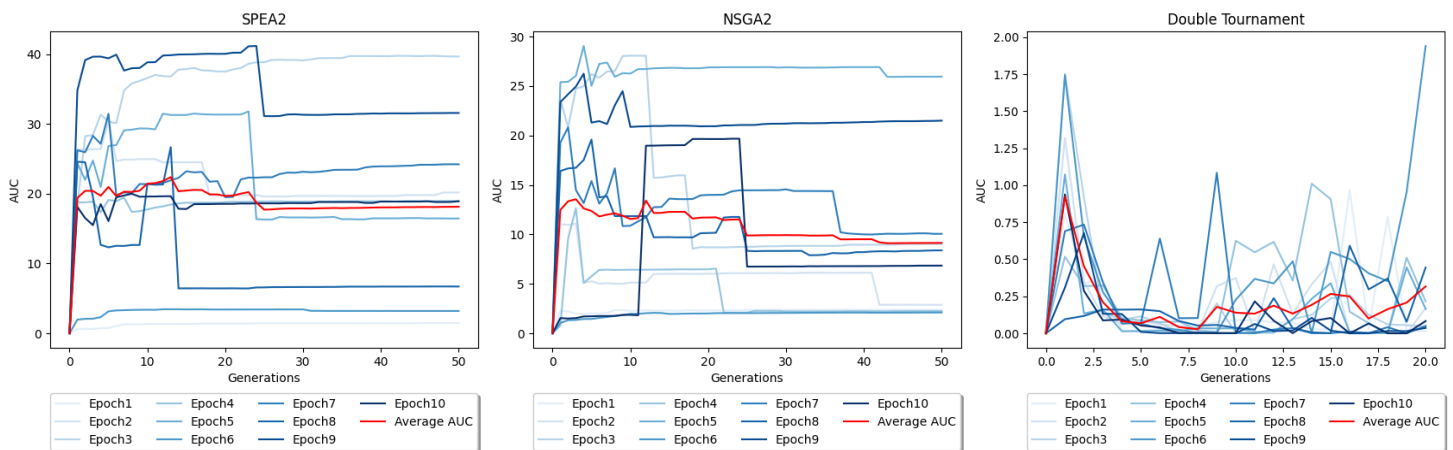
```
def mutate(individual):
    current_budget = budget - np.dot(np.array(individual), current_prices)
    adaptive_mutUniformInt(individual, low=0, indpb=0.1,
        current_budget=current_budget, push = 2)
    individual = check_negatives(individual)
    delta, power = 1, 1
    individual = check_budget_constraint(individual, delta, power)

    return individual,
```

Results

Step 1 – Selection Mechanisms

The results for three selection mechanisms are displayed below, ordered from the best performant to the worst. The area under the curve (AUC) of each epoch is coloured in tones of blue, and the average AUC is coloured in red. One epoch is one optimisation attempt, and each epoch starts with a different random initialisation of the population.



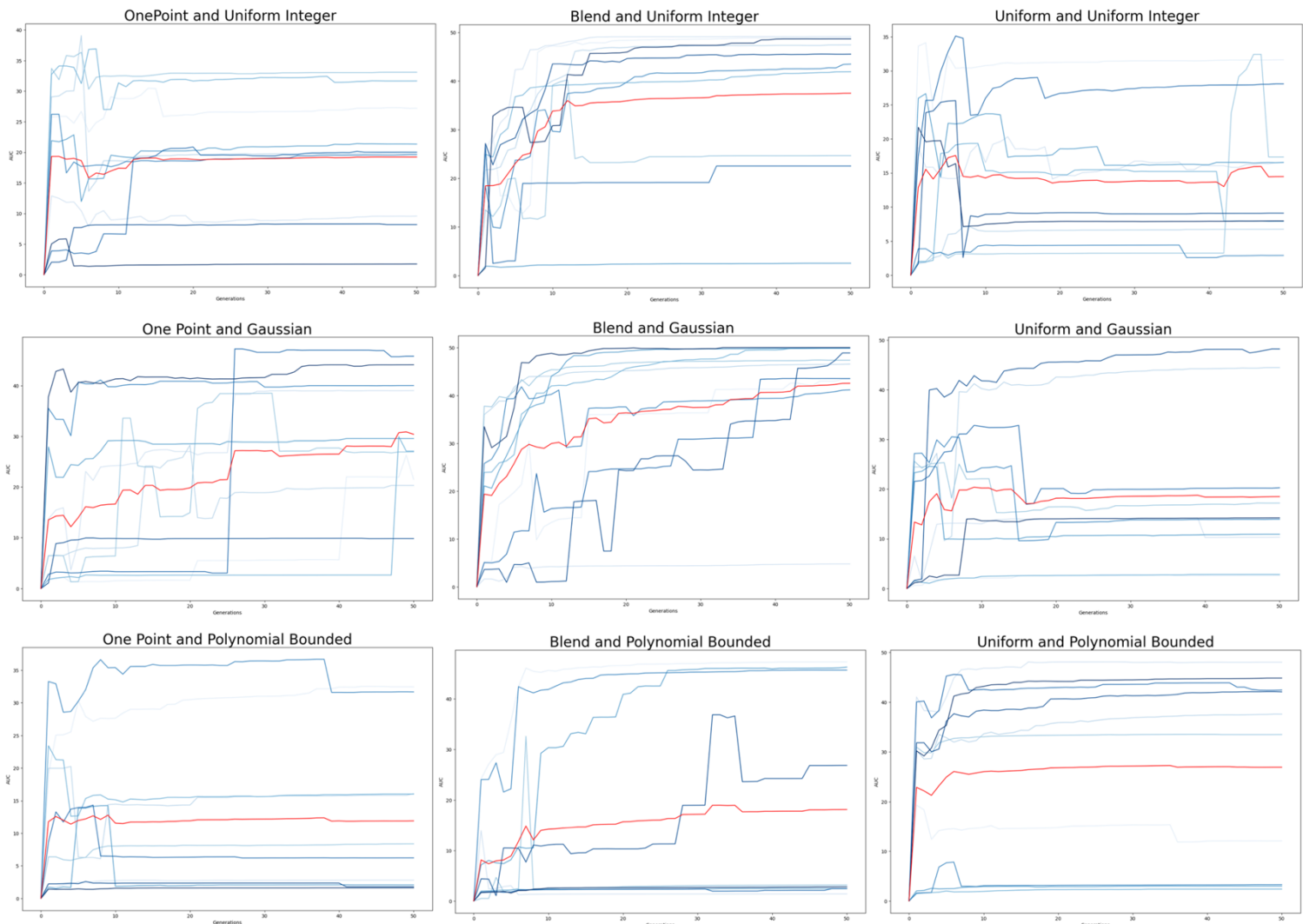
The best selection mechanism is SPEA2. On average it outperforms the other two methods, reaching AUCs and displaying greater stability across search. NSGA2 is the closest competitor, but it fails to reach values of AUC higher than 28 and often ends up in solutions that are worse than those found during the evolutionary process. Double Tournament selection is the worst, as it exhibits imperceptible optimisation improvements. To save time, its optimisation was run for 20 generations instead of 50.

Notably, SPEA2 also displays degrading behaviour during the evolutionary process, with 3 remarkable dips between generations 15 and 28 for epochs 5, 8, and 9. We hypothesise that this is due to the *push* parameter in the mutation mechanism. When set at 10, it caused many individuals to violate the budget constraint. This then caused the individuals to be passed through `check_budget_constraint` which resulted in deteriorating elite individuals. For robustness, we reduce *push* from 10 to 2 in step 2.

Step 2 – Crossover and Mutation Mechanisms

After choosing SPEA2, we tested the 9 different configurations of crossover and mutation mechanisms. The results are summarised and visualised below. Final AUC is the average area under the Pareto frontier of 10 epochs. Run time is the average amount of minutes per epoch to complete the optimisation.

Selection SPEA 2		Crossover		
		One Point	Blend	Uniform
Mutation	Uniform Int	Final AUC: 19.23 Run time: 7	Final AUC: 37.49 Run time: 16	Final AUC: 14.47 Run time: 10
	Gaussian	Final AUC: 30.41 Run time: 7	<u>Final AUC: 42.58</u> Run time: 17	Final AUC: 18 Run time: 10
	Polynomial	Final AUC: 11.91 <u>Run time: 5</u>	Final AUC: 18.11 Run time: 13	Final AUC: 26.92 Run time: 8



The configuration that yields the highest average AUC is Blend crossover with Gaussian mutation, which reaches a score of 42.58. The second highest is Blend Crossover and Uniform mutation with 37.49, followed by One Point crossover and Gaussian mutation with 30.41.

Blend crossover is the most effective crossover mechanism, achieving the highest score with two of the three mutation functions. With “luckier” runs, i.e. not getting stuck early in low-valued local maxima, Blend crossover may have also excelled with Polynomial Bounded mutation, as seen by the high AUCs that it reached in some epochs. Additionally, the AUC score grew continuously across generations only with Blend, meaning that decreasing *push* partly worked. In contrast, both One Point and Uniform crossover got stuck in local maxima around generation 12, as seen by the flattening of the AUC lines.

This success might be attributed to how Blend crossover creates offspring. In expectation, each gene is exactly in the middle of those of the two parents. This gives an adaptiveness-like property. As generations pass on, parents get closer to one another, which translates to smaller and more fine-tuned changes in the population.

Gaussian mutation appears to be the most effective mutation, achieving the highest score with two of the three crossover functions. Only when using Uniform crossover, Polynomial Bounded mutation managed to evolve the population away from local maxima. Similarly to Blend crossover, Gaussian mutation was more capable of improving the AUC along generations. In contrast, Uniform Integer and Polynomial Bounded mutation got stuck in local maxima around generation 10.

This success might be attributed to how Gaussian mutation applies a positive change to a gene with the same likelihood as it applies a negative change. This gives the chance to individuals to drop poor stocks and evolve towards better allocations. Notably, the Gaussian distribution plays a central role in more advanced algorithms called Estimation of Distributions Algorithms, such as Covariance Matrix Adaptation (CMA-ES). Conversely, Uniform Integer and Polynomial Bounded mutations have been configured to drop at most 0 stocks and thus lacked this reductionist exploration.

This contrast might indicate the existence of a completely different search methodology. Individuals could be initialised with very few stocks, such that portfolios are built from the group up. Intuitively, this optimisation would answer “What stocks and how much should I buy?” Instead, our approach has been more “aggressive”.

Individuals were initialised randomly but with a cost as close as possible to the budget constraint. The optimisation search then occurred between full portfolios, and thus answered “Which is the best portfolio?”. This overreliance on initialisation is likely the reason why we saw the same configuration of crossover and mutation function getting massively different results across epochs.

Future Research

Future research on this topic entails improvements on the evolutionary algorithm level and the problem level. On the algorithm level, a better strategy is needed to deal with individuals that exceed the budget constraint. The current approach of randomly decreasing some genes until the constraint is met risks erasing dominant solutions.

Elitism should also be prioritised more to avoid having evolutionary processes that move away from higher-valued solutions. Alternatively, a new initialisation algorithm strategy can be implemented and test different portfolio search approaches.

In terms of improvements related to just portfolio optimisation, some features could be added. For instance, optimisation could take into consideration side information about or preferences over the stocks. Furthermore, it would be interesting to research some methods for choosing solutions on the curve, such as the Sharpe Ratio.

Bibliography

1. Guarino, A., Santoro, D., Grilli, L. *et al.* EvoFolio: a portfolio optimization method based on multi-objective evolutionary algorithms. *Neural Comput & Applic* 36, 7221–7243 (2024). <https://doi.org/10.1007/s00521-024-09456-w>
2. Zhang, Byoung-Tak & Kim, Jung-Jib. (2001). Comparison of Selection Methods for Evolutionary Optimization. *Evolutionary Optimization*. 2.

Appendix

Pareto front curves obtained during some optimisations.

